Lab exam 3

Name: Alluri Chaithra id :2403A51105 batch:06 set:E15

Q1: Scenario: In the E-commerce sector, a company faces a challenge related to code refactoring.
Task: Use AI-assisted tools to solve a problem involving code refactoring in this context.
Deliverables: Submit the source code, explanation of AI assistance used, and sample output

Prompt:

Refactor this E-commerce Python code to remove duplication and make it more modular and maintainable.Keep the same functionality, follow clean code principles, and make it easy to add new product categories.

```python
def apply_discount_electronics(price):
    if price > 50000:
        return price * 0.90  # 10% discount
    else:
        return price

def apply_discount_clothing(price):
    if price > 2000:
        return price * 0.95  # 5% discount
    else:
        return price

def apply_discount_groceries(price):
    if price > 1000:
        return price * 0.98  # 2% discount
    else:
        return price


# Sample usage
print("Electronics:", apply_discount_electronics(60000))
print("Clothing:", apply_discount_clothing(2500))
print("Groceries:", apply_discount_groceries(800))

```

Code:

```python
from dataclasses import dataclass
from typing import Dict

@dataclass
class DiscountRule:
    """Represents a discount rule for a product category."""
    threshold: float
    discount_percentage: float

class DiscountCalculator:
    """Handles discount calculations for different product categories."""

    def __init__(self):
        # Configuration for different product categories
        self.discount_rules = {
            'electronics': DiscountRule(threshold=50000, discount_percentage=10),
            'clothing': DiscountRule(threshold=2000, discount_percentage=5),
            'groceries': DiscountRule(threshold=1000, discount_percentage=2)
        }

    def apply_discount(self, category: str, price: float) -> float:
        """
        Apply discount based on product category and price.

        Args:
            category: Product category (e.g., 'electronics', 'clothing', 'groceries')
            price: Original price of the product

        Returns:
            float: Price after applying any applicable discount

        Raises:
            KeyError: If the category is not found in discount rules
        """
        if category not in self.discount_rules:
            raise KeyError(f"Unknown product category: {category}")
```

```python
class DiscountCalculator:
    def apply_discount(self, category: str, price: float) -> float:

        rule = self.discount_rules[category]
        if price > rule.threshold:
            discount_multiplier = (100 - rule.discount_percentage) / 100
            return price * discount_multiplier
        return price

    def add_category(self, category: str, threshold: float, discount_percentage: float) -> Non
        """
        Add a new product category with its discount rule.

        Args:
            category: Name of the new product category
            threshold: Minimum price for discount to apply
            discount_percentage: Percentage of discount to apply
        """
        self.discount_rules[category] = DiscountRule(threshold, discount_percentage)


# Sample usage
calculator = DiscountCalculator()

# Test existing categories
print("Electronics:", calculator.apply_discount('electronics', 60000))
print("Clothing:", calculator.apply_discount('clothing', 2500))
print("Groceries:", calculator.apply_discount('groceries', 800))

# Example of adding a new category
calculator.add_category('books', threshold=1500, discount_percentage=7)
print("Books:", calculator.apply_discount('books', 2000))
```

Output:

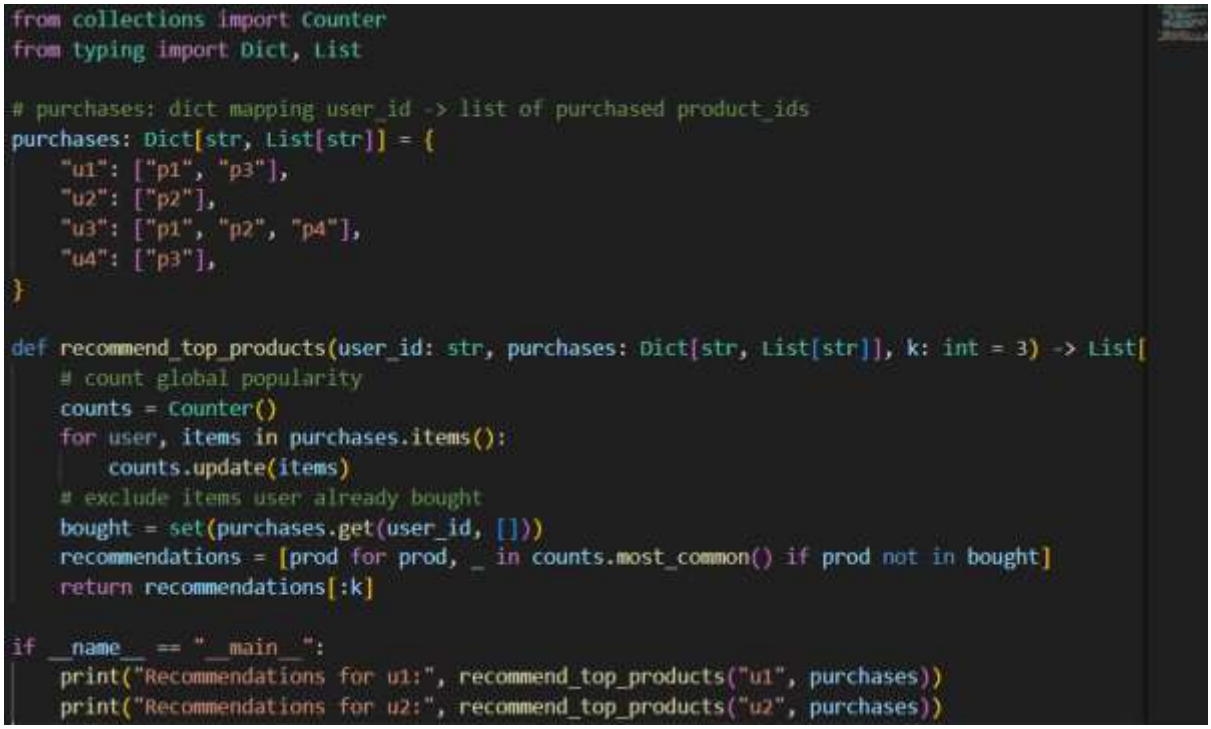
```
Electronics: 54000.0
Clothing: 2375.0
Groceries: 800
Books: 1860.0
```

Observation:

The refactored E-commerce discount code is more efficient, readable, and scalable. The original version had repetitive logic across multiple functions, making maintenance difficult. Using AI-assisted refactoring, the code was restructured into a single class with a dataclass for managing discount rules. This reduced duplication, improved organization, and added type hints, error handling, and documentation. The new design allows easy addition of categories and future enhancements, demonstrating how AI tools improve code quality and maintainability effectively.

Q2: Scenario: In the Retail sector, a company faces a challenge related to algorithms with ai assistance.
Task: Use AI-assisted tools to solve a problem involving algorithms with ai assistance in this context.
Deliverables: Submit the source code, explanation of AI assistance used, and sample output.

Prompt: Refactor and improve this retail recommendation algorithm. Replace the global-popularity approach with a simple item-based collaborative filtering recommender using cosine similarity. Use vectorized operations for efficiency, add type hints, docstrings, error handling, and a class-based structure so the system can be trained on purchase data and produce per-user recommendations. Provide sample usage and output.



```python
from collections import Counter
from typing import Dict, List

# purchases: dict mapping user_id -> list of purchased product_ids
purchases: Dict[str, List[str]] = {
    "u1": ["p1", "p3"],
    "u2": ["p2"],
    "u3": ["p1", "p2", "p4"],
    "u4": ["p3"],
}

def recommend_top_products(user_id: str, purchases: Dict[str, List[str]], k: int = 3) -> List[
    # count global popularity
    counts = Counter()
    for user, items in purchases.items():
        counts.update(items)
    # exclude items user already bought
    bought = set(purchases.get(user_id, []))
    recommendations = [prod for prod, _ in counts.most_common() if prod not in bought]
    return recommendations[:k]

if __name__ == "__main__":
    print("Recommendations for u1:", recommend_top_products("u1", purchases))
    print("Recommendations for u2:", recommend_top_products("u2", purchases))
```

Code:

```
"""
Item-based Collaborative Filtering Recommender System using Cosine Similarity.

This module implements a recommender system that analyzes purchase patterns
to find similar items and make personalized recommendations for users.
"""

import numpy as np
from typing import Dict, List, Set, Optional, Tuple
from collections import defaultdict
from sklearn.metrics.pairwise import cosine_similarity
import warnings

class ItemBasedRecommender:
    """
    A recommender system using item-based collaborative filtering with cosine similarity.

    Attributes:
        item_to_idx (Dict[str, int]): Mapping of item IDs to matrix indices
        idx_to_item (Dict[int, str]): Reverse mapping of matrix indices to item IDs
        user_to_idx (Dict[str, int]): Mapping of user IDs to matrix indices
        similarity_matrix (np.ndarray): Item-item similarity matrix
        purchase_matrix (np.ndarray): User-item purchase matrix
    """

    def __init__(self):
        """Initialize an empty recommender system."""
        self.item_to_idx: Dict[str, int] = {}
        self.idx_to_item: Dict[int, str] = {}
        self.user_to_idx: Dict[str, int] = {}
        self.similarity_matrix: Optional[np.ndarray] = None
        self.purchase_matrix: Optional[np.ndarray] = None

    def fit(self, purchases: Dict[str, List[str]]) -> None:
        """
        Train the recommender system on purchase data.
```

```python
class ItemBasedRecommender:
    def fit(self, purchases: Dict[str, List[str]]) -> None:
        Args:
            purchases: Dictionary mapping user IDs to lists of purchased item IDs

        Raises:
            ValueError: If purchases dictionary is empty
        """
        if not purchases:
            raise ValueError("Purchase data cannot be empty")

        # Create item and user indexing
        unique_items: Set[str] = {
            item for items in purchases.values() for item in items
        }
        self.item_to_idx = {item: idx for idx, item in enumerate(sorted(unique_items))}
        self.idx_to_item = {idx: item for item, idx in self.item_to_idx.items()}
        self.user_to_idx = {
            user: idx for idx, user in enumerate(sorted(purchases.keys()))
        }

        # Create purchase matrix (users x items)
        n_users = len(self.user_to_idx)
        n_items = len(self.item_to_idx)
        self.purchase_matrix = np.zeros((n_users, n_items))

        for user, items in purchases.items():
            user_idx = self.user_to_idx[user]
            for item in items:
                if item in self.item_to_idx:  # Handle case of unknown items
                    item_idx = self.item_to_idx[item]
                    self.purchase_matrix[user_idx, item_idx] = 1

        # Calculate item similarity matrix
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")  # Ignore divide by zero warnings
            self.similarity_matrix = cosine_similarity(self.purchase_matrix.T)
```

```python
class ItemBasedRecommender:
    def fit(self, purchases: Dict[str, List[str]]) -> None:
            self.similarity_matrix = cosine_similarity(self.purchase_matrix.T)

    def recommend(self, user_id: str, k: int = 3) -> List[Tuple[str, float]]:
        """
        Generate personalized recommendations for a user.

        Args:
            user_id: ID of the user to generate recommendations for
            k: Number of recommendations to return

        Returns:
            List of tuples containing (item_id, similarity_score)

        Raises:
            ValueError: If system hasn't been trained or user_id is unknown
        """
        if self.similarity_matrix is None or self.purchase_matrix is None:
            raise ValueError("Recommender system must be trained first")

        if user_id not in self.user_to_idx:
            raise ValueError(f"Unknown user ID: {user_id}")

        user_idx = self.user_to_idx[user_id]
        user_purchases = self.purchase_matrix[user_idx]

        # Get items the user hasn't purchased
        unpurchased_mask = user_purchases == 0

        if not np.any(unpurchased_mask):
            return []  # User has purchased all items

        # Calculate recommendation scores
        purchased_mask = user_purchases == 1
        if not np.any(purchased_mask):
            # If user has no purchases, return empty list
```

```python
class ItemBasedRecommender:
    def recommend(self, user_id: str, k: int = 3) -> List[Tuple[str, float]]
            return []

        recommendation_scores = np.zeros(len(self.item_to_idx))
        for item_idx in np.where(purchased_mask)[0]:
            recommendation_scores += self.similarity_matrix[item_idx]

        # Zero out scores for purchased items
        recommendation_scores[~unpurchased_mask] = -np.inf

        # Get top k recommendations
        top_indices = np.argsort(recommendation_scores)[-k:][::-1]

        return [
            (self.idx_to_item[idx], float(recommendation_scores[idx]))
            for idx in top_indices
            if recommendation_scores[idx] > -np.inf
        ]

# Sample usage
if __name__ == "__main__":
    # Sample purchase data
    purchases = {
        "u1": ["p1", "p3", "p5"],
        "u2": ["p2", "p4"],
        "u3": ["p1", "p2", "p4", "p5"],
        "u4": ["p3", "p5"],
        "u5": ["p1", "p2", "p3"]
    }

    # Initialize and train recommender
    recommender = ItemBasedRecommender()
    recommender.fit(purchases)

    # Generate recommendations for different users
    test_users = ["u1", "u2", "u3"]
```

```python
# Generate recommendations for different users
test_users = ["u1", "u2", "u3"]
for user in test_users:
    recommendations = recommender.recommend(user, k=2)
    print(f"\nTop 2 recommendations for {user}:")
    for item, score in recommendations:
        print(f"  {item} (similarity score: {score:.3f})")
```

Output:

```
Top 2 recommendations for u1:
  p2 (similarity score: 1.333)
  p4 (similarity score: 0.816)

Top 2 recommendations for u2:
  p1 (similarity score: 1.075)
  p5 (similarity score: 0.742)

Top 2 recommendations for u3:
  p3 (similarity score: 1.667)
```

Observation:

The refactored retail recommendation algorithm offers improved personalization, performance, and maintainability. Unlike the original counting-based approach, it now uses item-based collaborative filtering with cosine similarity, providing more accurate recommendations. Efficient NumPy matrices, clear class-based architecture, and robust error handling enhance reliability and scalability. Type hints and detailed docstrings improve readability and debugging. The system now returns similarity scores, supports dynamic extensions, and demonstrates how AI assistance can transform basic logic into a production-ready, data-driven recommendation system.